FIR Filter Accelerator - Hardware IP README

1. Functional Description

- **Feature 1: Parallel Block Processing** Instead of processing data one by one, this accelerator processes a "block" of multiple data points (e.g., 4 points) simultaneously in a single clock cycle. This significantly increases data throughput, making it highly efficient for high-speed signal processing.
- **Feature 2: Continuous Sliding Window Memory** The hardware automatically maintains an internal "sliding window" that remembers the necessary historical data from the previous clock cycle. When new data arrives, it is seamlessly stitched with the historical data to ensure the filtering process is mathematically continuous without any gaps.
- **Feature 3: "Configure-Once" Coefficient Locking** Once the filter coefficients are successfully loaded via the initial handshake, they are firmly locked in the hardware registers. The accelerator will continuously use these coefficients for all subsequent data streams until an explicit Reset signal is issued, saving bus bandwidth and software configuration overhead.

---

2. Top Module I/O Port Description

Port Name	Direction	Width	Description & Usage Notes (For Verification)
clk	Input	1	**System Clock.** All internal registers and state transitions are triggered on the rising edge (posedge).
rst_n	Input	1	**Hardware Reset.** Active-low, asynchronous reset. Must be pulled low at the very beginning of the simulation to initialize the FSM and pipeline registers.
start	Input	1	**MMIO Start Trigger.** Active-high. • Note: The internal logic includes an edge detector. A transition from 0 to 1 triggers the FSM from IDLE to BUSY. • CRITICAL: The FSM will stay in DONE and will not return to IDLE for the next task until this start signal is pulled back to 0 by the software/testbench.

Reset	Input	1	**MMIO Soft Reset.** Active-high. • Note: Asserting this clears the locked filter coefficients (coeff_loaded), flushes all historical data in the sliding window, and forces the FSM back to IDLE. Used to prepare the accelerator for a brand new set of coefficients.
AXS_S_fil_vld	Input	1	**Coefficient Stream Valid**. Asserted by the master when AXS_S_fil_data is valid.
AXS_S_fil_rdy	Output	1	**Coefficient Stream Ready.** • Note: This will remain 1 only when coefficients have NOT been loaded. Once a successful handshake occurs, this port drops to 0 and permanently locks until the MMIO Reset is asserted.
AXS_S_fil_data	Input	RAM_WIDTH	**Coefficient Data Bus.** **Packed format (e.g., {c3, c2, c1, c0}).** • Note: Must be **signed int16**. Loads all filter taps in a single beat.
AXS_S_fil_last	Input	1	**Coefficient Stream TLAST.** • Note: Not functionally required by the internal datapath since coefficients are loaded in one single beat, but included to comply with standard AXI-Stream protocol expectations.
AXS_S_src_vld	Input	1	**Source Data Stream Valid**. Asserted by the master (DMA) when input raw data is valid.
AXS_S_src_rdy	Output	1	**Source Data Stream Ready.** • Note: Asserted high only when the FSM is in the BUSY state and the internal pipeline is not stalled. Do not send data before start is triggered.
AXS_S_src_data	Input	RAM_WIDTH	**Source Data Bus.** **Packed format (e.g., {x3, x2, x1, x0}).** • Note: **Signed int16**. Must be properly scaled to avoid overflow

			after multiplication and addition.
AXS_S_src_last	Input	1	**Source Data Stream TLAST.** • CRITICAL: Must be asserted 1 precisely on the very last data beat (which should be the zero-padded flush data). This signal is tracked through the pipeline to cleanly terminate the BUSY state.
AXS_M_psum_vld	Output	1	**Output Result Stream Valid.** Asserted by the hardware when valid convolution results emerge from the 3-stage pipeline.
AXS_M_psum_rdy	Input	1	**Output Result Stream Ready.** Asserted by the downstream slave (DMA) to indicate it can accept data. • Note: Used for backpressure. If pulled low, the internal pipelines will hold their states.
AXS_M_psum_data	Output	RAM_WIDTH	**Output Result Bus. Packed format (e.g., {y3, y2, y1, y0}).** • Note: **Signed int16**. The first valid beat out of the hardware is a transient response (containing initial zeros) and should typically be discarded by the checker/scoreboard.
AXS_M_psum_last	Output	1	**Output Result Stream TLAST.** Hardware passes the AXS_S_src_last signal down the pipeline. Marks the absolute final block of output data.
busy	Output	1	**Status Flag.** High (1) when the FSM is actively processing data in the BUSY state.
done	Output	1	**Status Flag.** High (1) when the FSM has successfully processed the TLAST packet and entered the DONE state. • Note: Remains 1 until start is de-asserted or Reset is applied.

- **Hardware Parameter Constraint: The number of filter taps (Tap_Num)** is determined statically by the hardware parameters and **must strictly equal RAM_WIDTH / LEN_WIDTH.**

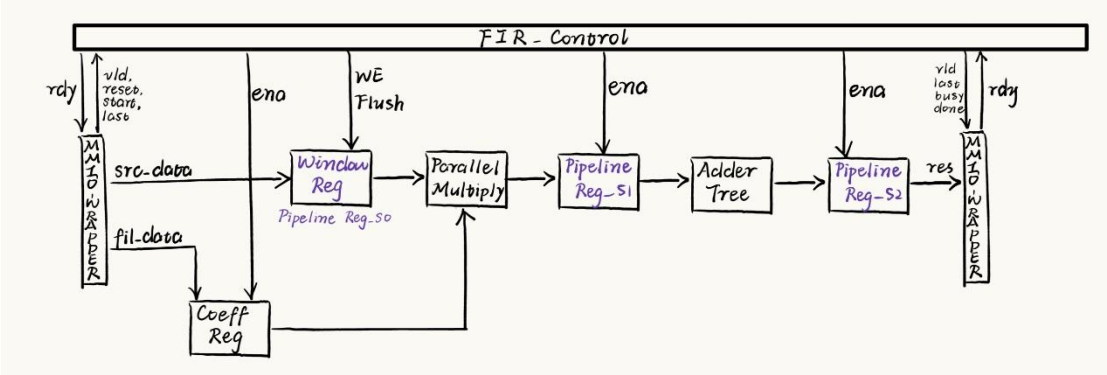
3. Computation Flow

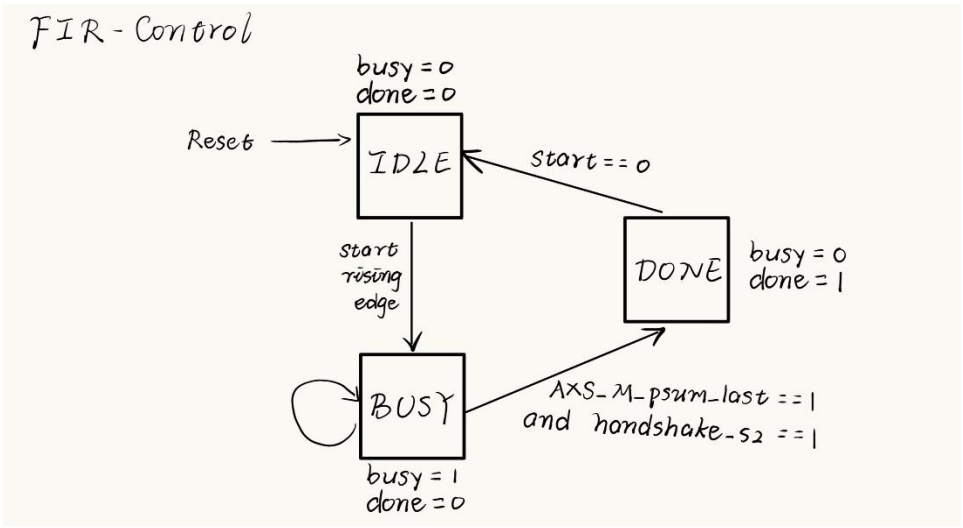
3.1 Input

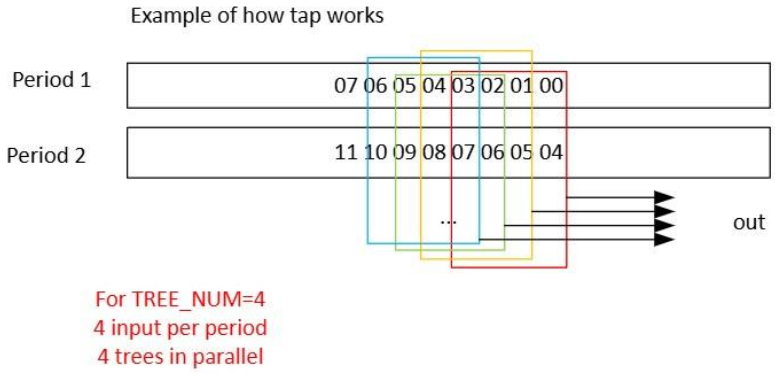
- **Data Types:** Signed 16-bit integers (int16).
- **Storage & Protocol:** Data is transferred via the AXI-Stream protocol. Multiple int16 words are packed into a single wide bus (e.g., a 64-bit bus carries four 16-bit words: {Data[3], Data[2], Data[1], Data[0]}).

3.2 Process

- **Step 1 (Pipeline S0 - Sliding Window Update):**
    - *Input:* Raw AXI-Stream source data block.
    - *Process:* The hardware receives the new data block upon a successful handshake and shifts it into the internal sliding window register, combining it with the historical data from the previous beat.
    - *Output:* A concatenated wide data vector containing both old and new data points.
- **Step 2 (Pipeline S1 - Parallel Multiplication):**
    - *Input:* The concatenated data vector from S0 and the locked filter coefficients.
    - *Process:* Performs a massive parallel multiplication using the filter taps against the sliding window data array.
    - *Output:* An array of 32-bit multiplication results (internally truncated/kept to 16 bits).
- **Step 3 (Pipeline S2 - Adder Tree Compression):**
    - *Input:* The array of multiplied results from S1.
    - *Process:* Parallel balanced, recursive binary adder trees sum up the respective products for each output data point.
    - *Output:* The final filtered result block.
- **Hardware Architecture Diagram:**







3.3 Output

- **Data Format:** The output stream shares the exact same packed format and bit-width as the input (e.g., four int16 results packed into a 64-bit word: result = {result[3], result[2], result[1], result[0]}). Transmitted via the Master AXI-Stream protocol.

---

4. Important Usage Notes & Constraints (CRITICAL)

To guarantee correct hardware execution and data extraction, the software/verification

environment **MUST** adhere to the following rules:

1. **Discard Initial Transient Output:** Because the internal sliding window is initialized with zeros, the very first output block produced by the hardware represents a transient response. **This first valid output group must be discarded/ignored by the receiving software.**
2. **State Machine Recovery (DONE to IDLE):** Once the hardware finishes a task and enters the DONE state, it will **only** return to the IDLE state if the start signal is de-asserted (pulled to 0) or if the Reset signal is asserted.
3. **Coefficient Retention & Clearing:** The filter coefficients are "Configure-Once". If you wish to clear them or write a new set of coefficients, you **must assert the Reset signal high**. Otherwise, the hardware will persistently use the previously loaded coefficients for all subsequent AXI-Stream transactions.
4. **Arithmetic Range Limitation:** All input data and coefficients are in signed int16 format. The input data values **must be reasonably scaled**. The datapath relies on natural bit-truncation; therefore, any addition or multiplication that causes an overflow beyond 16 bits will result in data corruption.
5. **Zero-Padding and Tail Data Extraction (Flushing):** Due to the sliding window architecture, the final valid calculations are left inside the pipeline. The software must pad the end of the stream based on the alignment of the total data count:
    - **If the total data count is a multiple of 4:** You must append a completely empty dummy block ({0, 0, 0, 0}) at the end and assert TLAST with it. For the final output block generated by this flush, **only the lowest 16 bits (Data[0])** represent the last valid filtered calculation.
    - **If the total data count is NOT a multiple of 4:** You must pad the remaining Most Significant Bits (MSBs) of the final data block with zeros, assert TLAST, and selectively extract the valid outputs from the final generated block:
        - If the last block has **3 valid inputs** (highest 16 bits are 0): All **4** outputs in the final block are valid.
        - If the last block has **2 valid inputs** (highest 32 bits are 0): Only the lower **48 bits (3 outputs)** of the final block are valid.
        - If the last block has **1 valid input** (highest 48 bits are 0): Only the lower **32 bits (2 outputs)** of the final block are valid.